PostgreSQL ALTER TABLE + CASE

10 Combined Medium & Hard Interview Practice Questions

Topics covered: Add columns • Remove columns • Rename columns • Change data types • Default values • Constraints • Rename table • CASE expressions

Starter Table

```
CREATE TABLE employees (
employee_id SERIAL PRIMARY KEY,
first_name VARCHAR(50),
last_name VARCHAR(50),
department VARCHAR(30),
salary INTEGER,
experience_years INTEGER,
performance_score INTEGER,
city VARCHAR(50),
employment_status VARCHAR(20)
);

INSERT INTO employees
(first_name, last_name, department, salary, experience_years, performance_score, city, employment_status)
VALUES
('Rahul', 'Sharma', 'IT', 75000, 4, 8, 'Mumbai', 'Active'),
('Priya', 'Patil', 'HR', 55000, 2, 7, 'Pune', 'Active'),
('Amit', 'Verma', 'IT', 120000, 8, 9, 'Mumbai', 'Active'),
('Sneha', 'Joshi', 'Finance', 65000, 3, 6, 'Thane', 'Active'),
('Rohan', 'Mehta', 'IT', 45000, 1, 5, 'Nashik', 'Probation'),
('Neha', 'Kulkarni', 'HR', 90000, 6, 9, 'Mumbai', 'Active'),
('Vikas', 'Shah', 'Finance', 110000, 7, 8, 'Pune', 'Active'),
('Anjali', 'Deshmukh', 'IT', 60000, 2, 7, 'Thane', 'Inactive');
```

Q1 — Add column + CASE

Add a new column **bonus** with an appropriate numeric data type. Then write a query displaying employee_id, first_name, salary and bonus. Use CASE to calculate bonus:
• Salary >= 100000 → 15% of salary
• Salary >= 70000 → 10%
• Salary >= 50000 → 7%
• Otherwise → 5%
Do not update the bonus column yet; first calculate it using CASE.

Q2 — Rename column + CASE

Rename **performance_score** to **performance_rating**. Then display first_name, performance_rating and performance_category.
CASE rules: 9–10 = Excellent; 7–8 = Good; 5–6 = Average; below 5 = Poor.

Q3 — Add column + DEFAULT + CASE

Add **job_level** with a suitable data type and set its DEFAULT to 'Entry'. Then use CASE to display the calculated level:
• Experience >= 8 → Senior
• Experience >= 5 → Mid-Level
• Experience >= 2 → Junior

• Otherwise → Entry
Output: employee_id, first_name, experience_years, job_level, calculated_level.
Interview thought: What is the difference between DEFAULT and a CASE-calculated value?

Q4 — Change data type + CASE

Change salary from INTEGER to a suitable DECIMAL/NUMERIC type. Then display first_name, salary and salary_range:
• >= 100000 → High Salary
• >= 70000 → Above Average
• >= 50000 → Average
• Otherwise → Low Salary.

Q5 — Add constraint + CASE

Add a constraint ensuring **performance_rating** is between 1 and 10. Then display first_name, performance_rating and performance_status:
• 9–10 → Top Performer
• 7–8 → Good Performer
• 5–6 → Needs Improvement
• 1–4 → Poor Performer
Think: Why do the constraint and CASE solve two different problems?

Q6 — Multiple CASE conditions + ALTER

Add **employee_grade**. Classify employees using both salary and performance:
• salary >= 100000 AND performance >= 9 → A+
• salary >= 80000 AND performance >= 8 → A
• salary >= 60000 AND performance >= 7 → B
• salary >= 50000 AND performance >= 6 → C
• otherwise → D
Display first_name, salary, performance_rating and employee_grade.
Interview challenge: Why can changing WHEN order produce different results?

Q7 — Remove column + CASE

Remove the **city** column. Then categorize employees using employment status and experience:
• Active + experience >= 5 → Experienced Active Employee
• Active + experience < 5 → Junior Active Employee
• Inactive → Inactive Employee
• Probation → Probation Employee
Output: first_name, experience_years, employment_status, employee_category.

Q8 — Rename table + multiple CASE expressions

Rename **employees** to **company_employees**. Then write one SELECT containing two CASE expressions.
Salary category: >=100000 High; >=70000 Medium-High; >=50000 Medium; otherwise Low.
Experience category: >=7 Veteran; >=4 Experienced; >=2 Junior; otherwise Fresher.
Output: employee_id, first_name, salary, salary_category, experience_years, experience_category.

Q9 — ALTER + CASE + business logic

Add **promotion_status**. Determine eligibility:
• performance >= 8 AND experience >= 3 AND employment_status = Active → Eligible for Promotion
• otherwise → Not Eligible

Display first_name, experience_years, performance_rating, employment_status, promotion_status.
Extra challenge: Add promotion_priority:
• performance >=9 AND experience >=5 → High Priority
• performance >=8 AND experience >=3 → Medium Priority
• otherwise → Low Priority.

Q10 — Final Interview Challenge

Perform these tasks in order on **company_employees**:
1. Add **annual_bonus NUMERIC**.
2. Add a constraint ensuring annual_bonus >= 0.
3. Rename **job_level** to **employee_level**.
4. Write a complex query displaying: employee_id, first_name, department, salary, performance_rating, experience_years, employee_level, bonus_percentage, bonus_amount, employee_status.
Bonus percentage: performance >=9 AND salary >=100000 → 20%; performance >=8 AND salary >=80000 → 15%; performance >=7 AND salary >=60000 → 10%; otherwise → 5%.
Employee status: Inactive → Not Eligible; Active + performance >=8 + experience >=5 → Promotion Ready; Active + performance >=7 → Performing Well; Active → Needs Review; Probation → Under Evaluation.
Bonus amount: salary × bonus_percentage.

Difficulty & Skill Map

Question	Level	Main skills
Q1	Medium	ADD COLUMN + CASE
Q2	Medium	RENAME COLUMN + CASE
Q3	Medium	DEFAULT + CASE
Q4	Medium	ALTER TYPE + CASE
Q5	Medium	CONSTRAINT + CASE
Q6	Hard	Multiple conditions
Q7	Hard	DROP COLUMN + CASE
Q8	Hard	RENAME TABLE + multiple CASE
Q9	Hard	AND + CASE + business logic
Q10	Very Hard	Multiple ALTER + multiple CASE

Practice Strategy

For every question, first identify: (1) which ALTER operation is required, (2) which column(s) are affected, (3) the CASE conditions, (4) the correct WHEN order, and (5) the expected output. Avoid looking up syntax immediately. Focus on the logic first.

Goal: Strengthen PostgreSQL schema-modification skills and CASE/business-logic thinking for coding interviews.